Informe de Arquitectura y Patrones de Diseño

Sistema de Transcripción de Voz *SpeechNotes*

Análisis y Diseño de Software
ESPE — VI Nivel SII 2025

11 de febrero de 2026

Índice

1. Introducción **3**
- 1.1. Contexto . . . 3
- 1.2. Problema . . . 3
- 1.3. Objetivos del Proyecto . . . 3
  - 1.3.1. Objetivo General . . . 3
  - 1.3.2. Objetivos Específicos . . . 3

2. Materiales y Métodos **4**
- 2.1. Stack Tecnológico . . . 4
- 2.2. Arquitectura General . . . 4

3. Resultados: Patrones de Diseno Identificados **5**
- 3.1. Patrones Creacionales . . . 5
  - 3.1.1. Singleton — `ApiClient` (Frontend) . . . 5
  - 3.1.2. Factory Method — `FormatterFactory` . . . 6
  - 3.1.3. Abstract Factory — `TranscriptionEnvironmentFactory` . . . 7
  - 3.1.4. Lazy Initialization — Transcriber . . . 8
- 3.2. Patrones Estructurales . . . 9
  - 3.2.1. Adapter — `AudioProcessorPort` . . . 9
  - 3.2.2. Facade — `register_socket_events` y `ApiClient` . . . 10
  - 3.2.3. Repository — `TranscriptionRepository` . . . 12
- 3.3. Patrones Comportamentales . . . 15
  - 3.3.1. Strategy — `VADStrategy` . . . 15
  - 3.3.2. State — Máquina de Estados VAD . . . 17
  - 3.3.3. Observer — Socket.IO Events . . . 18
  - 3.3.4. Dependency Injection — FastAPI `Depends()` . . . 20
  - 3.3.5. Service Layer — `TranscriptionService` . . . 21
  - 3.3.6. Template Method — Pipeline de Audio . . . 22

4. Resumen de Patrones y Principios SOLID **25**
- 4.1. Tabla Resumen de Patrones . . . 25
- 4.2. Adherencia a Principios SOLID . . . 25

5. Discusión **26**
5.1. Fortalezas Arquitectónicas 26
5.2. Áreas de Mejora 26

6. Conclusiones **26**

1. Introducción

1.1. Contexto

SpeechNotes es un sistema de transcripción de voz en tiempo real que convierte audio capturado desde el navegador en texto formateado, utilizando el motor de reconocimiento de voz **NVIDIA Riva**. El sistema opera como una aplicación web completa con un frontend construido en **Next.js/React** y un backend en **FastAPI + Socket.IO**, respaldado por **MongoDB** para la persistencia de datos.

1.2. Problema

La captura, procesamiento y almacenamiento de transcripciones de audio en tiempo real presenta desafíos en:

- **Múltiples formatos de audio** que deben ser normalizados.
- **Detección inteligente de voz** (VAD) para segmentar frases.
- **Comunicación bidireccional** entre navegador y servidor.
- **Formateo inteligente** del texto transcrito usando modelos de IA.
- **Persistencia y consulta** eficiente de grandes volúmenes de datos.

1.3. Objetivos del Proyecto

1.3.1. Objetivo General

Desarrollar un sistema de transcripción de voz en tiempo real escalable y modular, que permita la captura, procesamiento y almacenamiento eficiente de audio desde navegadores web, integrando motores de reconocimiento de voz avanzados y técnicas de formateo inteligente mediante Inteligencia Artificial.

1.3.2. Objetivos Específicos

1. **Diseñar una arquitectura robusta** basada en capas, que facilite la separación de responsabilidades entre el frontend, backend y servicios de procesamiento de audio.
2. **Implementar un pipeline de audio eficiente** que normalice múltiples formatos de entrada y utilice estrategias de detección de actividad de voz (VAD) para optimizar el uso de recursos y mejorar la precisión de la transcripción.
3. **Desarrollar un sistema de comunicación en tiempo real** bidireccional mediante WebSockets, asegurando una baja latencia en la transmisión de audio y la recepción de transcripciones.
4. **Aplicar patrones de diseño de software** (Creacionales, Estructurales y Comportamentales) para garantizar la mantenibilidad, extensibilidad y calidad del código fuente.
5. **Integrar capacidades de Inteligencia Artificial** para el post-procesamiento y formateo automático del texto transcrito, mejorando su legibilidad y utilidad para el usuario final.

2. Materiales y Métodos

2.1. Stack Tecnológico

Capa	Tecnología	Rol
Frontend	Next.js 14 + React 18	Interfaz de usuario
Backend	FastAPI + Socket.IO	API REST y comunicación en tiempo real
Motor STT	NVIDIA Riva	Reconocimiento de voz
Base de datos	MongoDB	Persistencia documental
IA	Kimi K2-Thinking	Formateo inteligente de notas
Audio	FFmpeg + pydub	Conversión y normalización de audio

Cuadro 1: Stack tecnológico del sistema SpeechNotes.

2.2. Arquitectura General

El sistema sigue una **arquitectura por capas** (Layered Architecture) con separación clara entre:

1. **Presentation Layer**: Dashboard React con hooks personalizados.
2. **API Layer**: Routers FastAPI (REST) + Socket.IO (WebSocket).
3. **Service Layer**: Lógica de negocio encapsulada.
4. **Repository Layer**: Acceso a datos (MongoDB).
5. **Domain/Core**: Factories, formatters, configuración.

La estructura detallada de estos componentes y sus interacciones se muestra en la figura 1 a continuación, donde se aprecia la integración de los servicios externos con el core de la aplicación.

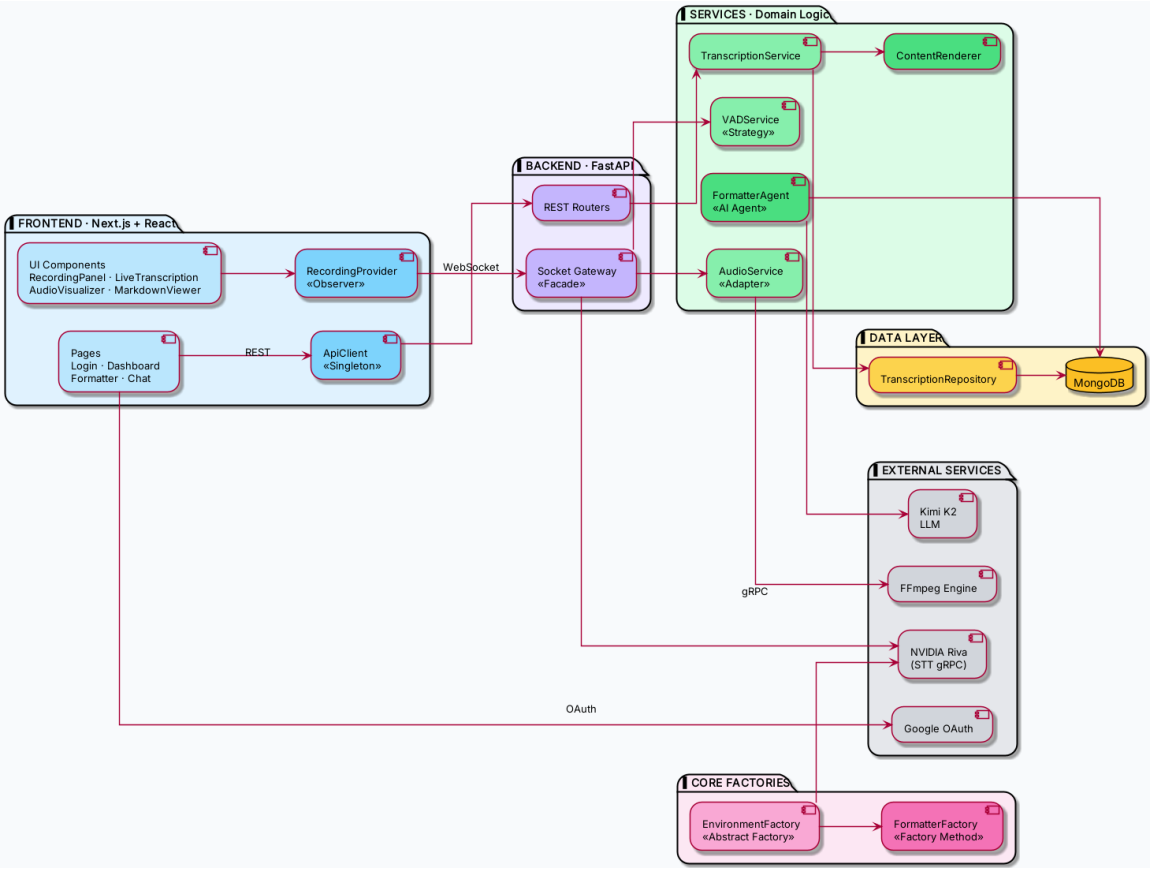


Figura 1: Arquitectura Clean Architecture + Microservicios de SpeechNotes.

3. Resultados: Patrones de Diseno Identificados

A continuación se presentan los patrones identificados, agrupados según la clasificación clásica de la *Gang of Four* (GoF).

3.1. Patrones Creacionales

3.1.1. Singleton — `ApiClient` (Frontend)

Intención: Garantizar que una clase tenga una única instancia y proporcionar un punto de acceso global a ella.

Implementación: La clase `ApiClient` del frontend almacena su instancia en un atributo estático `instance` y expone el método `getInstance()`. Esto asegura que todas las llamadas a la API compartan la misma configuración, caché y lógica de reintentos.

web/services/ApiClient.ts -- Singleton

```
export class ApiClient {
  private static instance: ApiClient;
  private baseUrl = '/api';

  private constructor() {}

  public static getInstance(): ApiClient {
    if (!ApiClient.instance) {
```

```
      ApiClient.instance = new ApiClient();
    }
    return ApiClient.instance;
  }
}
```

Justificación: Evita instancias duplicadas del cliente HTTP, centraliza la caché en memoria y los headers compartidos, tal como se representa en el diagrama de clases de la figura 2 a continuación.

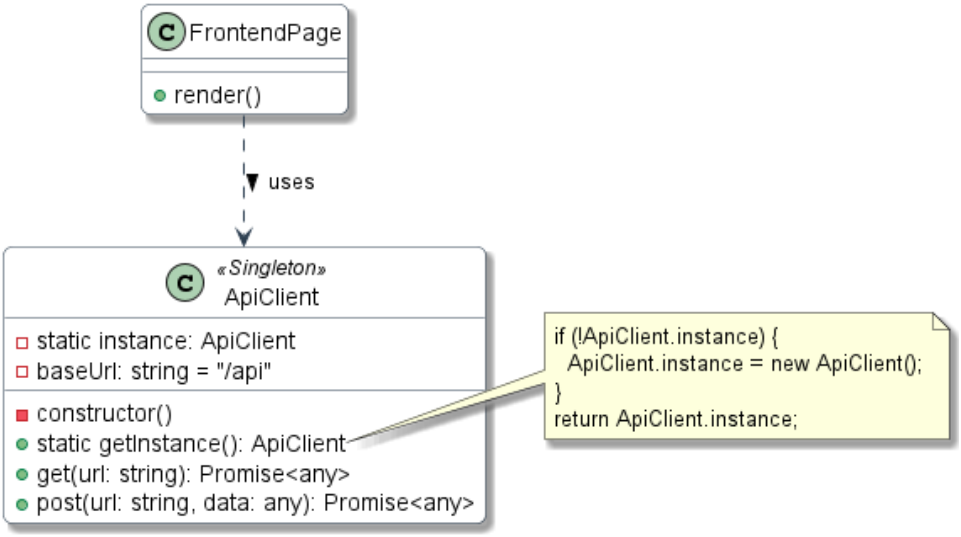


Figura 2: Diagrama de clases: Singleton (ApiClient).

3.1.2. Factory Method — `FormatterFactory`

Intención: Definir una interfaz para crear un objeto, pero permitir que las subclases decidan qué clase instanciar.

src/transcription/formatters.py -- Factory Method

```
class FormatterFactory:
    """Factory Pattern: Centralized creation"""

    _formatters = {
        'markdown': MarkdownFormatter,
        'segmented_markdown':
            SegmentedMarkdownFormatter,
        'plain': PlainTextFormatter,
    }

    @classmethod
    def create(cls, format_type='markdown'):
        formatter_class = cls._formatters.get(
            format_type
        )
        if not formatter_class:
            raise ValueError(
                f"Unknown: {format_type}"
            )
        return formatter_class()
```

Principios SOLID:

- **OCP**: Para agregar un nuevo formato basta con crear la clase y agregarla al diccionario.
- **DIP**: El código cliente depende de la abstracción `OutputFormatter`, no de las clases concretas.

La estructura de esta factoría se detalla en la figura 3 que se muestra a continuación.

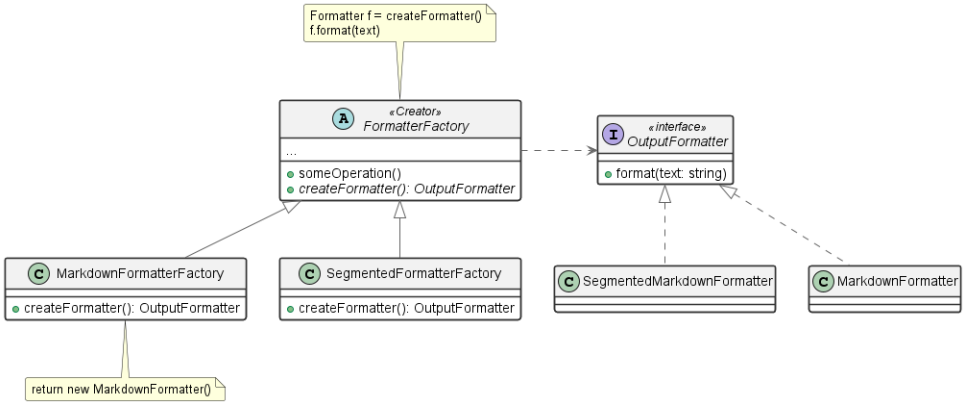


Figura 3: Diagrama de clases: Factory Method.

3.1.3. Abstract Factory — `TranscriptionEnvironmentFactory`

Intención: Proveer una interfaz para crear familias de objetos relacionados sin especificar sus clases concretas.

src/core/environment_factory.py -- Abstract Factory

```python
class TranscriptionEnvironmentFactory(ABC):
    @abstractmethod
    def create_transcriber(self):
        pass

    @abstractmethod
    def create_recorder(self,
            recorder_type,
            audio_config=None,
            vad_config=None):
        pass

    @abstractmethod
    def create_formatter(self):
        pass

class RivaLiveFactory(
    TranscriptionEnvironmentFactory
):
    def create_transcriber(self):
        if self._transcriber is None:
            cfg = self.config_manager\
                .get_riva_config()
            self._transcriber = \
                RivaTranscriber(cfg)
        return self._transcriber

    def create_formatter(self):
        return SegmentedMarkdownFormatter()

class LocalBatchFactory(
    TranscriptionEnvironmentFactory
```

```
):
    def create_formatter(self):
        return MarkdownFormatter()
```

Justificación: Cada "entorno" (Riva Live, Local Batch) necesita componentes compatibles entre sí. La Abstract Factory garantiza que no se mezclen componentes incompatibles. La jerarquía de clases para esta implementación se ilustra en la figura 4 a continuación.

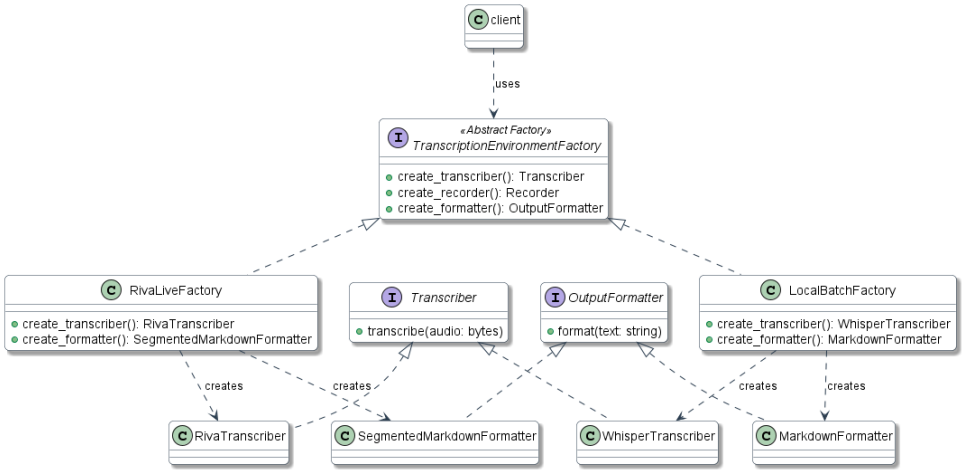


Figura 4: Diagrama de clases: Abstract Factory.

3.1.4. Lazy Initialization — Transcriber

Intención: Retrasar la creación de un objeto costoso hasta que sea realmente necesario.

backend/services/socket_handler.py -- Lazy Init

```
transcriber = None

def get_transcriber():
    """Lazy Initialization - Creational"""
    global transcriber
    if transcriber is None:
        try:
            factory = (
                FactoryProvider.get_riva_live()
            )
            transcriber = (
                factory.create_transcriber()
            )
        except Exception as e:
            transcriber = None
    return transcriber
```

Este mecanismo de inicialización tardía para el motor de transcripción se esquematiza en la figura 5 a continuación.

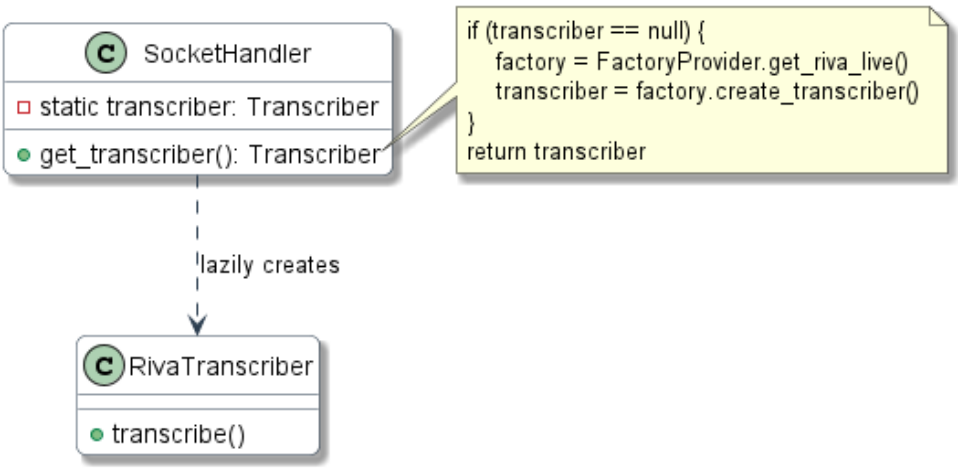


Figura 5: Diagrama de clases: Lazy Initialization (Transcriber).

3.2. Patrones Estructurales

3.2.1. Adapter — `AudioProcessorPort`

Intención: Convertir la interfaz de una clase en otra interfaz que los clientes esperan. Permite que clases con interfaces incompatibles trabajen juntas.

backend/services/audio_service.py -- Adapter

```python
class AudioProcessorPort(ABC):
    """Adapter Pattern interface"""

    @abstractmethod
    def to_pcm(self, raw_data: bytes)
            -> bytes:
        """Convert to PCM 16kHz."""
        ...

    @abstractmethod
    def get_format_name(self)
            -> str:
        ...

class WebMAudioAdapter(AudioProcessorPort):
    """Adapts WebM/Opus from browser to PCM"""

    def to_pcm(self, raw_data: bytes)
            -> bytes:
        from pydub import AudioSegment
        audio = AudioSegment.from_file(
            tmp_path, format="webm"
        )
        audio = audio.set_channels(1)
        audio = audio.set_sample_width(2)
        audio = audio.set_frame_rate(16000)
        return audio.raw_data

class PCMPassthroughAdapter(AudioProcessorPort):
    """Identity adapter: no conversion"""

    def to_pcm(self, raw_data: bytes)
            -> bytes:
        return raw_data
```

Principios SOLID:

- **OCP**: Se pueden agregar adaptadores (e.g., WAV, FLAC) sin modificar el código existente.
- **DIP**: `SocketHandler` depende de `AudioProcessorPort`, no de las implementaciones concretas.
- **SRP**: Cada adaptador solo es responsable de una conversión.

El diseño del adaptador para el procesamiento de audio se visualiza en la figura 6 a continuación.

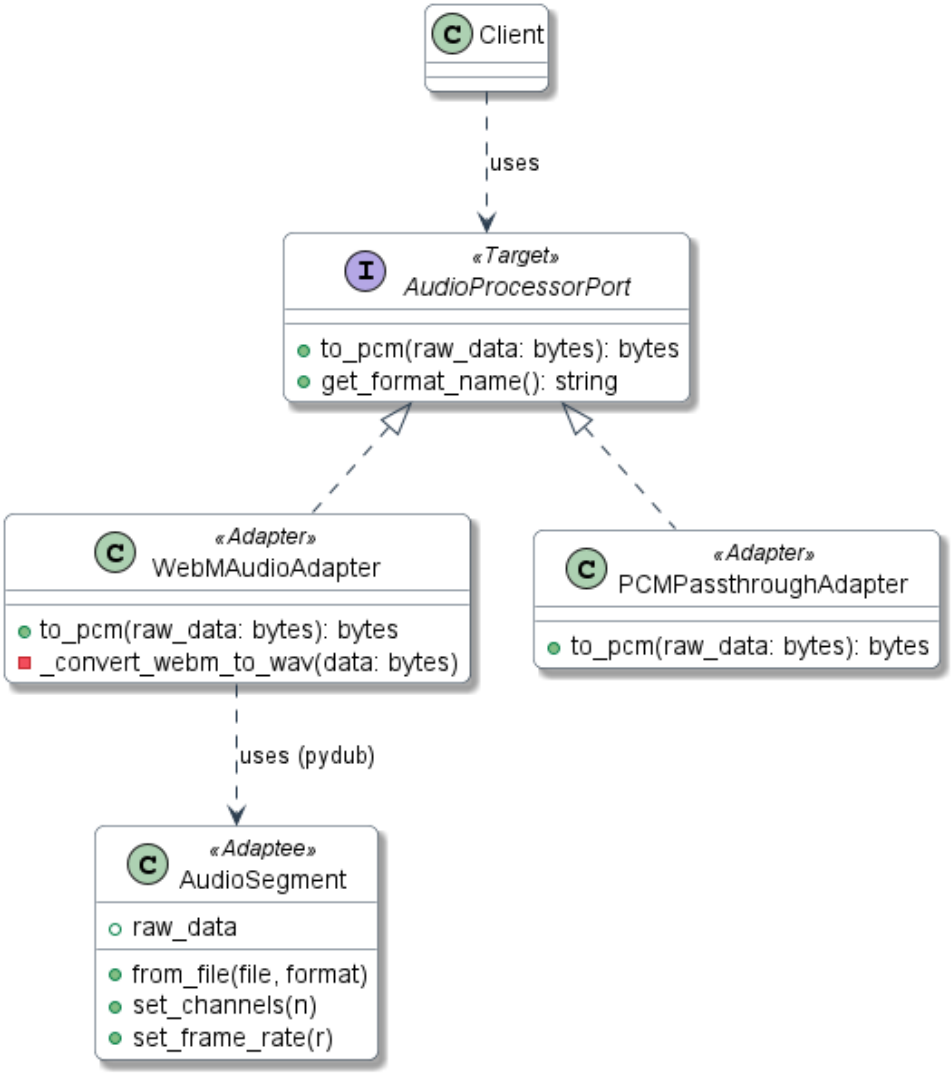


Figura 6: Diagrama de clases: Adapter (AudioProcessorPort).

3.2.2. Facade — `register_socket_events` y `ApiClient`

Intención: Proporcionar una interfaz unificada a un conjunto de interfaces de un subsistema, haciéndolo más fácil de usar.

Backend (Facade):

backend/services/socket_handler.py -- Facade

```python
def register_socket_events(sio):
    """
    Facade Pattern entry point.
    Delegates to:
      - AudioProcessorPort (Adapter)
      - VADStrategy (Strategy)
      - Riva transcriber
      - TranscriptionRepository
    """
    @sio.event
    async def connect(sid, environ):
        pass # session initialization

    @sio.event
    async def audio_chunk(sid, data):
        pcm = await asyncio.to_thread(
            _webm_adapter.to_pcm, data
        )
        await _process_pcm_chunk(
            sio, sid, pcm, session
        )
```

Frontend (Facade): `ApiClient` oculta la complejidad de `fetch()`, headers, reintentos, caché y manejo de errores.

web/services/ApiClient.ts -- Facade

```typescript
private async request<T>(
  endpoint: string,
  options?: RequestInit,
  retries = 3
): Promise<T> {
  const url = this.baseUrl + endpoint;
  let lastError;
  for (let i = 0; i < retries; i++) {
    const ctrl = new AbortController();
    const tid = setTimeout(
      () => ctrl.abort(), 15000
    );
    try {
      const res = await fetch(url, {
        ...options,
        headers,
        signal: ctrl.signal,
      });
      clearTimeout(tid);
      return res.json();
    } catch (error: any) {
      clearTimeout(tid);
      lastError = error;
    }
  }
  throw lastError;
}
```

La simplificación de la interfaz del subsistema a través del manejador de sockets se muestra en el diagrama de la figura 7 a continuación.

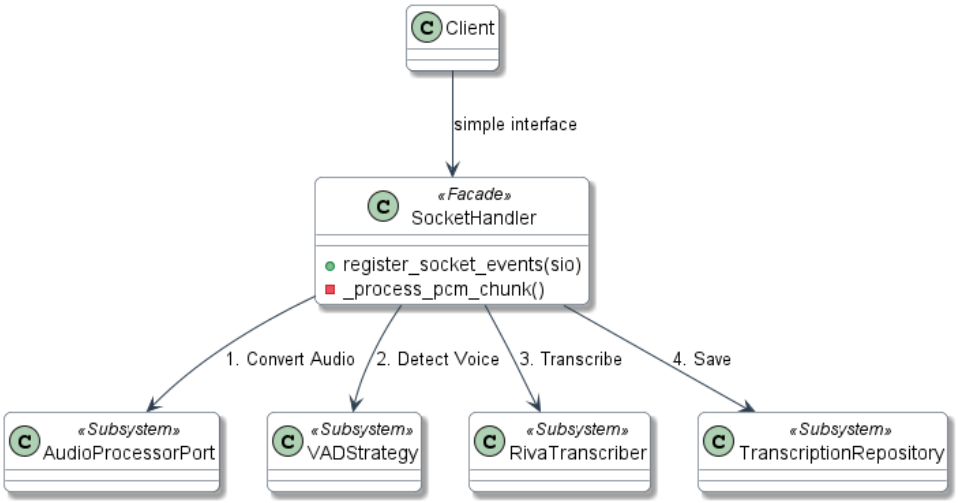


Figura 7: Diagrama de clases: Facade (SocketHandler).

3.2.3. Repository — `TranscriptionRepository`

Intención: Mediar entre la capa de dominio y la capa de datos, actuando como una colección de objetos en memoria.

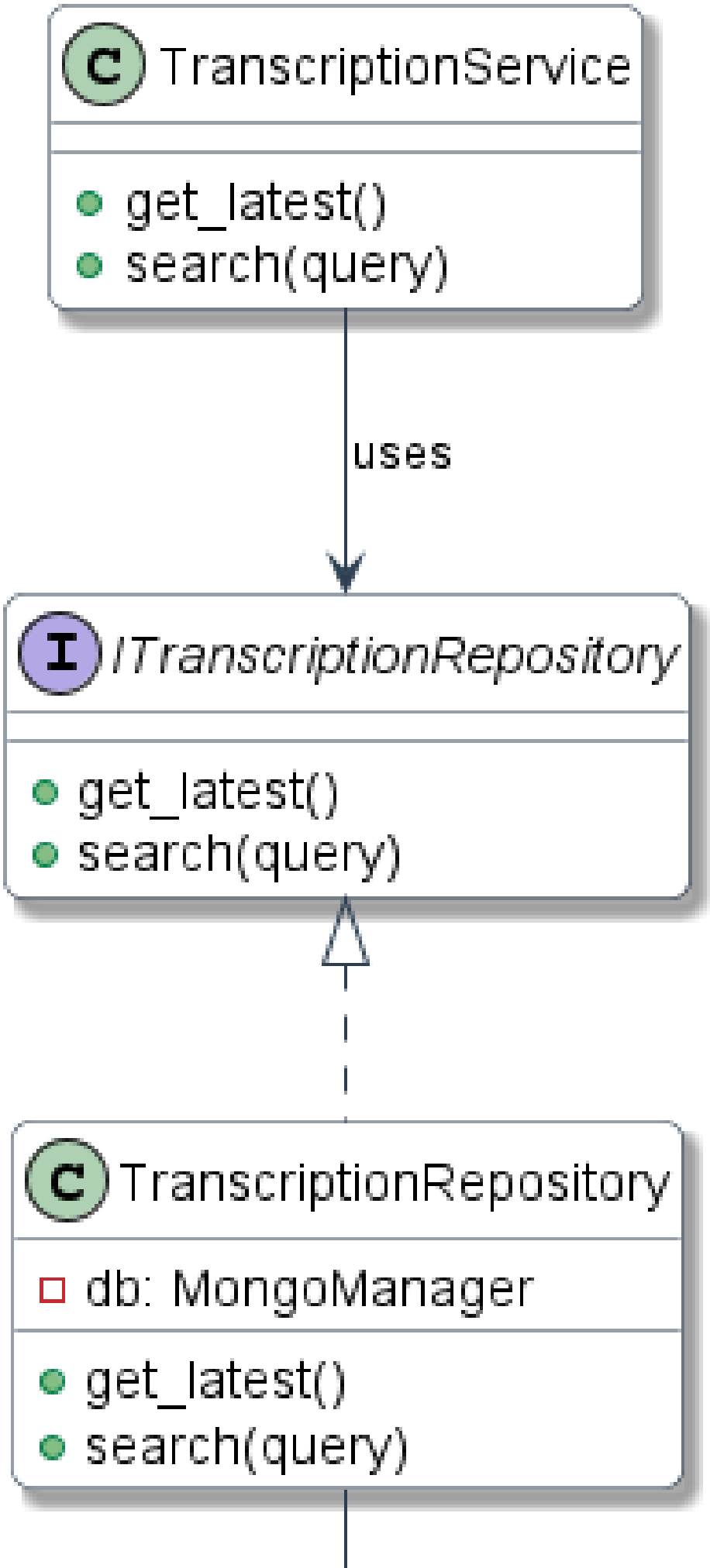
backend/repositories/transcription_repository.py

```python
class TranscriptionRepository:
    """Repository Pattern for persistence"""

    def __init__(self):
        self.db = MongoManager()
        self.generator = DocumentGenerator()

    def get_latest(self):
        doc = self.db.transcriptions.find_one(
            {"deleted": {"$ne": True}},
            sort=[("created_at", -1)]
        )
        if doc:
            return self._process_document(doc)
        return None

    def search(self, query):
        pipeline = [
            {"$match": {
                "$text": {"$search": query},
                "deleted": {"$ne": True}
            }},
            {"$sort": {
                "score": {"$meta": "textScore"}
            }},
            {"$limit": 20}
        ]
        results = list(
            self.db.transcriptions.aggregate(
                pipeline
            )
        )
        return {"items": [
            self._process_document(r)
            for r in results
        ]}
```

Principios SOLID:

- **SRP**: Solo responsable de acceso a datos.
- **DIP**: `TranscriptionService` depende de la abstracción del repositorio, no de MongoDB directamente.

La mediación entre la capa de dominio y el almacenamiento persistente se esquematiza en el diagrama de la figura 8 a continuación.


TranscriptionService
get_latest()
search(query)
uses
ITranscriptionRepository
get_latest()
search(query)
TranscriptionRepository
db: MongoManager
get_latest()
search(query)

3.3. Patrones Comportamentales

3.3.1. Strategy — `VADStrategy`

Intención: Definir una familia de algoritmos, encapsular cada uno y hacerlos intercambiables, de modo que el algoritmo pueda variar de forma independiente respecto a los clientes que lo utilizan.

Problema que resuelve: En un sistema de transcripción en tiempo real, la **Detección de Actividad de Voz** (VAD, *Voice Activity Detection*) es un componente crítico que determina cuándo el usuario está hablando y cuándo hay silencio. Sin embargo, existen múltiples algoritmos para realizar esta tarea, cada uno con diferentes ventajas:

- **Threshold-based (RMS)**: rápido y ligero, calcula la energía de la señal de audio y la compara contra umbrales configurables. Ideal para entornos con poco ruido ambiental.
- **Silero VAD**: un modelo de Machine Learning preentrenado que ofrece mayor precisión en entornos ruidosos, pero requiere más recursos computacionales.
- **WebRTC VAD**: algoritmo nativo del navegador, útil como complemento para detección en el lado del cliente.

Sin el patrón Strategy, el código del `SocketHandler` tendría que contener toda la lógica de todos los algoritmos de VAD en un solo método monolítico con condicionales (`if/elif/else`), lo cual viola el **Principio de Abierto/Cerrado (OCP)** y dificulta la incorporación de nuevos algoritmos.

Solución adoptada: Se define una **clase abstracta** `VADStrategy` que establece el contrato común mediante dos métodos: `process_chunk()` para analizar cada fragmento de audio, y `reset()` para reiniciar el estado interno. Cada estrategia concreta implementa su propio algoritmo:

backend/services/vad_service.py -- Strategy

```python
class VADStrategy(ABC):
    """Abstract Strategy for VAD"""

    @abstractmethod
    def process_chunk(
            self, pcm_data: bytes):
        ...

    @abstractmethod
    def reset(self):
        ...

class ThresholdVADStrategy(VADStrategy):
    """Threshold-based VAD using RMS"""

    def __init__(self, config=None):
        self.config = config or VADConfig()
        self._state = VADState.IDLE
        self._silence_counter = 0

    def process_chunk(
            self, pcm_data: bytes):
        rms = AudioUtils.calculate_rms(
```

```
            pcm_data
        )
        should_buffer = False
        phrase_ended = False

        if self._state == VADState.IDLE:
            if rms > self.config\
                    .voice_threshold:
                self._state = VADState.SPEAKING
                should_buffer = True

        return VADResult(
            state=self._state, rms=rms,
            should_buffer=should_buffer,
            phrase_ended=phrase_ended
        )
```

Justificación y beneficios:

- **Intercambiabilidad**: el `SocketHandler` trabaja con cualquier implementación de `VADStrategy` sin conocer los detalles internos del algoritmo utilizado.
- **Extensibilidad**: para agregar un nuevo algoritmo de VAD (por ejemplo, Silero VAD basado en redes neuronales), basta con crear una nueva subclase de `VADStrategy` sin modificar ninguna línea del código existente.
- **Configurabilidad**: cada estrategia recibe su propia configuración (`VADConfig`), permitiendo umbrales y parámetros personalizados por usuario o sesión.
- **Testabilidad**: se pueden crear estrategias *mock* para pruebas unitarias sin necesidad del hardware de audio real.

Principios SOLID:

- **OCP**: agregar nuevas estrategias no requiere modificar el código existente del handler.
- **DIP**: `SocketHandler` depende de la abstracción `VADStrategy`, no de las implementaciones concretas.
- **SRP**: cada estrategia encapsula únicamente su algoritmo de detección; la decisión de qué hacer con el resultado reside en el handler.

El desacoplamiento logrado mediante este patrón se observa en el diagrama de clases de la figura 9 a continuación, donde se aprecia cómo el contexto (`SocketHandler`) delega la lógica de detección a la jerarquía de estrategias.

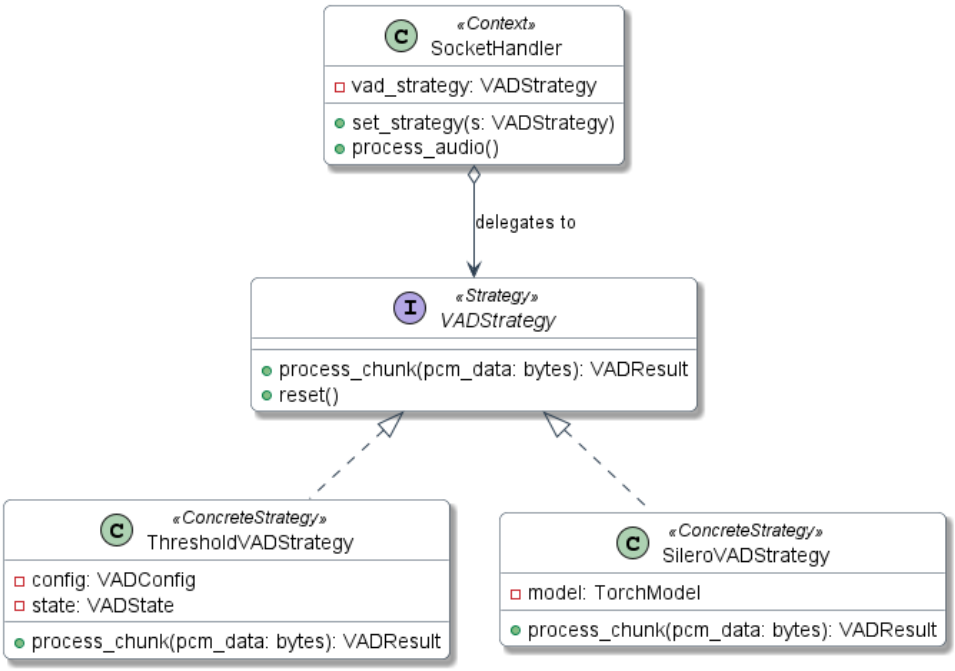


Figura 9: Diagrama de clases: Strategy — La interfaz `VADStrategy` permite intercambiar algoritmos de detección de voz sin afectar al `SocketHandler`.

3.3.2. State — Máquina de Estados VAD

Intención: Permitir que un objeto modifique su comportamiento cuando su estado interno cambia.

La clase `ThresholdVADStrategy` implementa internamente una máquina de estados con tres estados posibles:

Estado actual	Condición	Estado siguiente
`IDLE`	RMS > voice_threshold	`SPEAKING`
`SPEAKING`	RMS < silence_threshold	`SILENCE_AFTER_SPEECH`
`SILENCE_AFTER_SPEECH`	N chunks silenciosos	`IDLE` (phrase_ended)
`SILENCE_AFTER_SPEECH`	RMS > voice_threshold	`SPEAKING`

La **histéresis** (umbrales diferentes para onset y offset) previene el *toggling* rápido entre estados. El flujo de transiciones de esta máquina de estados se detalla en la figura 10 a continuación.

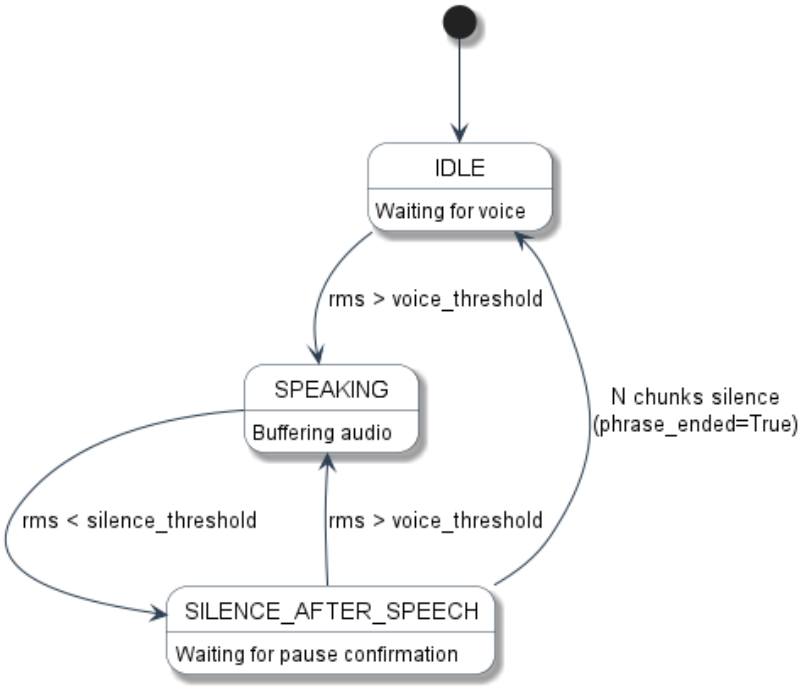


Figura 10: Diagrama de estados: Máquina de Estados VAD.

3.3.3. Observer — Socket.IO Events

Intención: Definir una dependencia uno-a-muchos entre objetos, de modo que cuando uno cambie de estado, todos sus dependientes sean notificados y actualizados automáticamente.

Problema que resuelve: SpeechNotes opera como un sistema de **comunicación bidireccional en tiempo real** donde múltiples clientes (navegadores web) envían flujos continuos de audio al servidor. El servidor debe reaccionar a diferentes **tipos de eventos** de cada cliente de forma simultánea:

- `connect`: un nuevo cliente se conecta; se debe inicializar su sesión, asignarle una estrategia VAD y preparar los buffers de audio.
- `audio_chunk`: el cliente envía un fragmento de audio; se debe convertir a PCM, analizar con VAD y, si corresponde, enviar al motor de transcripción.
- `stop_recording`: el cliente detiene la grabación; se debe procesar el audio pendiente, generar la transcripción final y guardarla en la base de datos.
- `disconnect`: el cliente se desconecta; se deben liberar los recursos asociados a su sesión.

Sin el patrón Observer, el servidor tendría que implementar un bucle de *polling* que consulte constantemente si hay nuevos datos de cada cliente, lo que sería ineficiente y no escalaría con múltiples usuarios concurrentes. Además, el código que produce los eventos estaría fuertemente acoplado al código que los consume.

Solución adoptada: Socket.IO implementa nativamente el patrón Observer a través de su sistema de eventos. El servidor **(Subject)** emite eventos y los **handlers (Observers)** se suscriben a ellos mediante el decorador `@sio.event`. Cada handler es independiente y se ejecuta automáticamente cuando ocurre su evento correspondiente:

socket_handler.py ~~ Observer

```python
@sio.event
async def connect(sid, environ):
    """Observer: handles 'connect' event
    Initializes session with VAD strategy
    and audio buffers for the new client."""
    active_sessions[sid] = {
        "transcription_buffer": [],
        "vad_strategy":
            ThresholdVADStrategy(vad_cfg),
    }

@sio.event
async def audio_chunk(sid, data):
    """Observer: handles 'audio_chunk'
    Converts audio and processes through
    the VAD+transcription pipeline."""
    pcm = await asyncio.to_thread(
        _webm_adapter.to_pcm, data
    )
    await _process_pcm_chunk(
        sio, sid, pcm, session
    )

@sio.event
async def stop_recording(sid):
    """Observer: handles 'stop_recording'
    Finalizes recording, processes pending
    audio, and persists transcription."""
    asyncio.create_task(
        _handle_stop_recording(sid)
    )
```

Justificación y beneficios:

- **Desacoplamiento**: el emisor del evento (cliente WebSocket) no conoce ni depende de los handlers que procesan el evento. Esto permite agregar nuevos handlers sin modificar el código del cliente.
- **Escalabilidad**: Socket.IO gestiona automáticamente las conexiones concurrentes, permitiendo que múltiples usuarios graben audio simultáneamente sin interferencia entre sesiones.
- **Asincronía**: combinado con `asyncio`, cada handler se ejecuta de forma no bloqueante, lo que maximiza el rendimiento del servidor al procesar audio de múltiples clientes en paralelo.
- **Eventos tipados**: cada evento tiene un nombre claro (`connect`, `audio_chunk`, `stop_recording`), lo que facilita la trazabilidad y depuración del flujo de datos.

Principios SOLID:

- **SRP**: cada handler (*observer*) es responsable de una única acción específica.

- **OCP**: se pueden agregar nuevos eventos y handlers sin modificar los existentes; por ejemplo, agregar un evento `change_model` para cambiar el motor de transcripción en caliente.
- **ISP**: cada handler implementa solo la interfaz que necesita (un único método asíncrono por evento), sin depender de funcionalidades que no utiliza.

La arquitectura de suscripción y notificación de eventos de Socket.IO se esquematiza en el diagrama de la figura 11 a continuación, donde se muestra cómo el servidor (*Subject*) notifica a los handlers registrados (*Observers*) cuando ocurre cada tipo de evento.

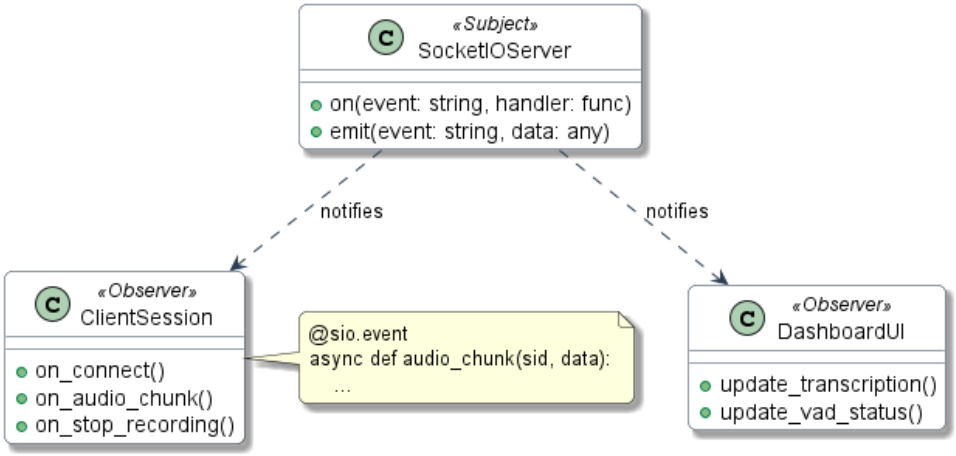


Figura 11: Diagrama de clases: Observer — El `SocketIOServer` (Subject) notifica automáticamente a los handlers suscritos cuando se producen eventos como `connect`, `audio_chunk` y `stop_recording`.

3.3.4. Dependency Injection — FastAPI `Depends()`

Intención: Invertir el control de la creación de dependencias, inyectándolas desde el exterior.

backend/routers/transcriptions.py -- DI

```python
def get_service():
    """Dependency provider.
    Override in tests to inject mocks."""
    return TranscriptionService(
        repository=
            TranscriptionRepository(),
        renderer=
            ContentRenderer()
    )

@router.get("/latest")
async def get_latest_transcription(
    service = Depends(get_service)
):
    return service.get_latest()
```

Principio DIP: El router no crea directamente sus dependencias; FastAPI las inyecta. En tests, se puede sustituir `get_service` por un mock sin modificar el router. El mecanismo de inyección se ilustra en la figura 12 a continuación.

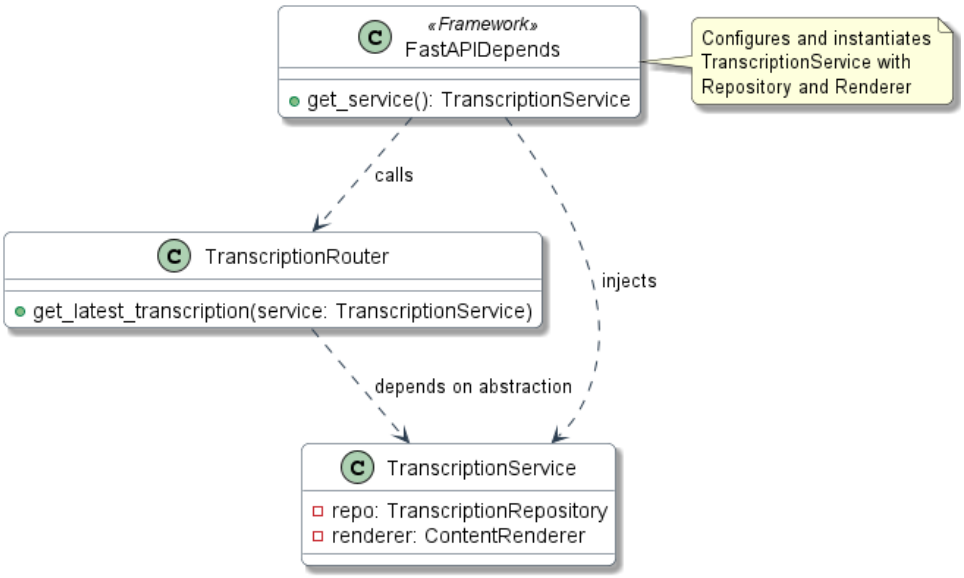


Figura 12: Diagrama de clases: Dependency Injection.

3.3.5. Service Layer — `TranscriptionService`

Intención: Definir la frontera de la aplicación con una capa de servicios que coordine la respuesta.

backend/services/transcription_service.py

```python
class TranscriptionService:
    """Service Layer: business logic"""

    def __init__(self, repository, renderer):
        self.repo = repository
        self.renderer = renderer

    def get_latest(self):
        doc = self.repo.get_latest()
        if doc:
            doc["rendered_content"] = (
                self.renderer.render(
                    doc.get("content", "")
                )
            )
        return doc

    def search(self, query):
        results = self.repo.search(query)
        return results.get("items", [])
```

La organización de la lógica de negocio en esta capa se visualiza en la figura 13 que se muestra a continuación.

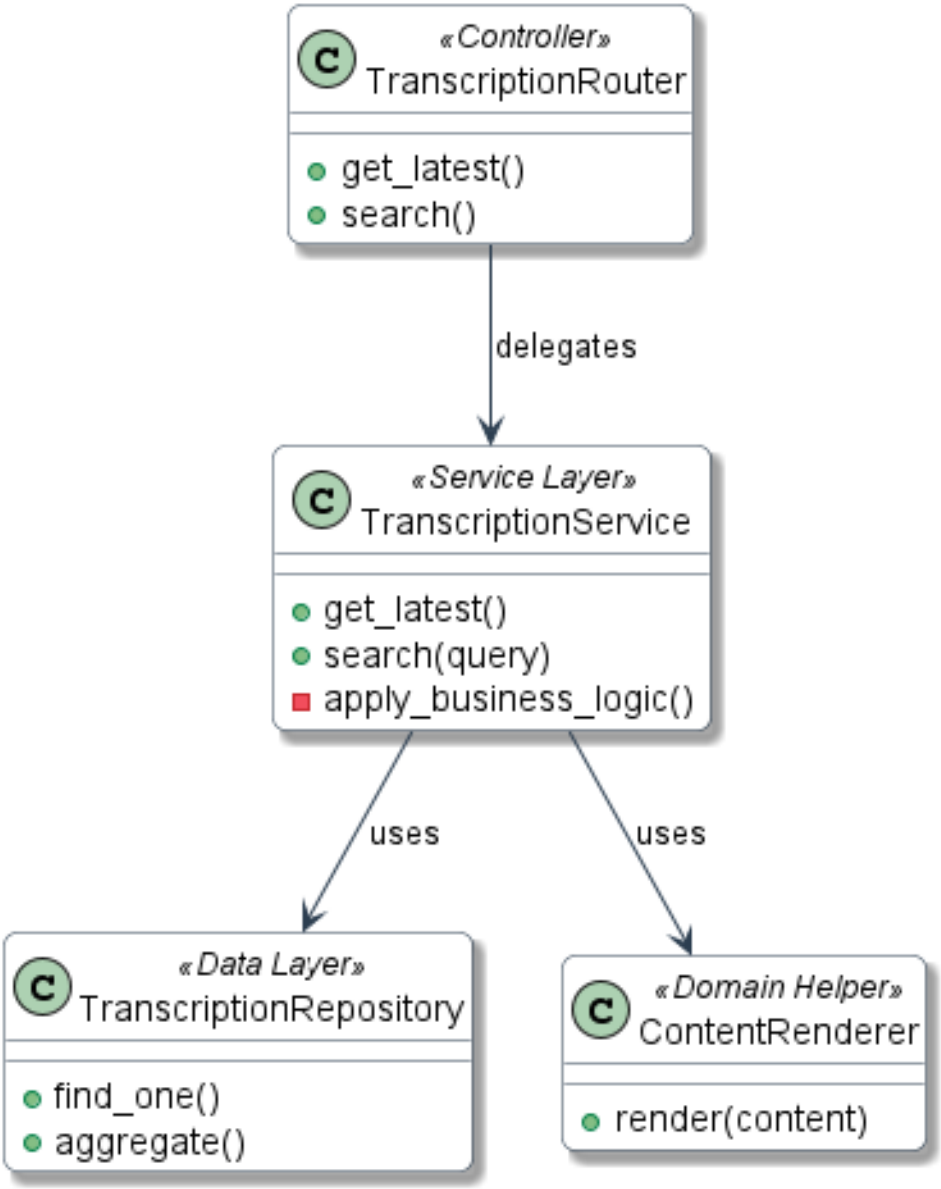


Figura 13: Diagrama de clases: Service Layer.

3.3.6. Template Method — Pipeline de Audio

Intención: Definir el esqueleto de un algoritmo en una operación, delegando algunos pasos a las subclases.

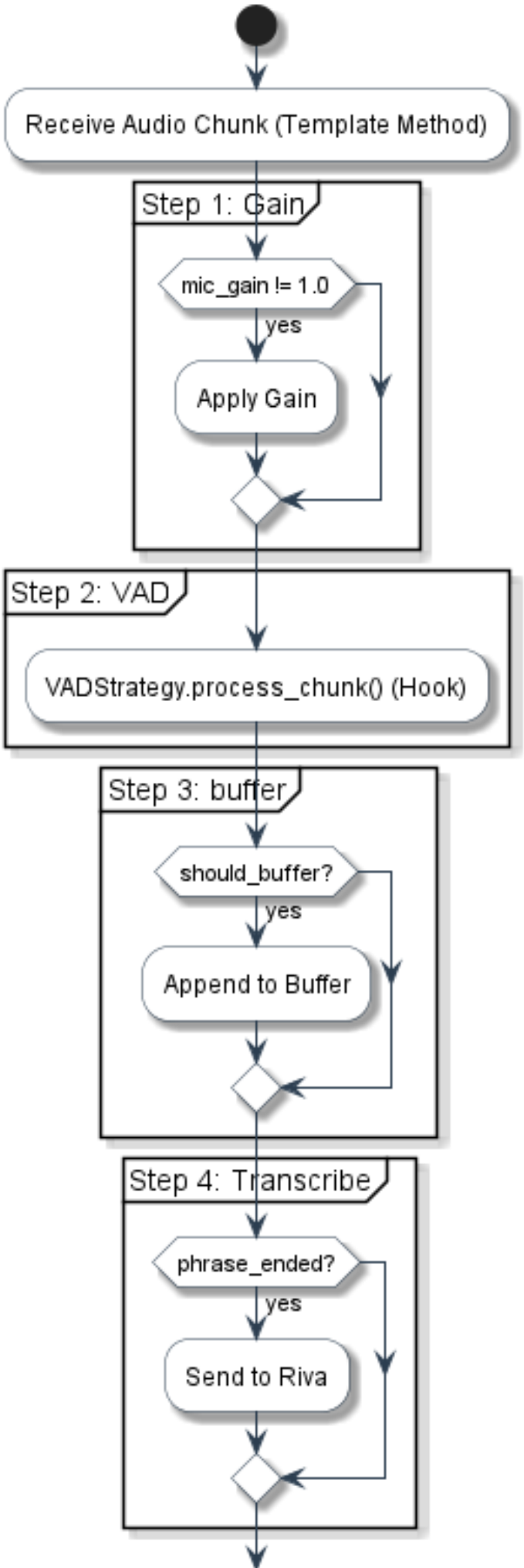
El método `_process_pcm_chunk` define un pipeline fijo de 4 pasos:

1. **Gain**: Aplicar ganancia de micrófono.
2. **VAD**: Delegar a `VADStrategy`.
3. **Buffer**: Acumular PCM según la decisión del VAD.
4. **Transcribe**: Enviar buffer al motor Riva cuando `phrase_ended = True`.

socket_handler.py -- Template Method

```python
async def _process_pcm_chunk(
    sio, sid, pcm, session
):
    # Step 1: Apply gain
    if mic_gain != 1.0:
        pcm = AudioUtils.apply_gain(
            pcm, mic_gain
        )

    # Step 2: VAD delegation
    vad = session["vad_strategy"]
    result = vad.process_chunk(pcm)

    # Step 3: Buffer logic
    if result.should_buffer:
        session["speech_buffer"] += pcm

    # Step 4: Transcribe
    if result.phrase_ended:
        text = await asyncio.to_thread(
            transcribe_audio_chunk,
            session["speech_buffer"], 0
        )
```

El flujo secuencial definido para el procesamiento de cada bloque de audio se muestra en el diagrama de actividad de la figura 14 presentada a continuación.


Receive Audio Chunk (Template Method)
Step 1: Gain
mic_gain != 1.0
yes
Apply Gain
Step 2: VAD
VADStrategy.process_chunk() (Hook)
Step 3: buffer
should_buffer?
yes
Append to Buffer
Step 4: Transcribe
phrase_ended?
yes
Send to Riva

4. Resumen de Patrones y Principios SOLID

4.1. Tabla Resumen de Patrones

Categoría	Patrón	Archivo	Clase/Función
Creacional	Singleton	`ApiClient.ts`	`ApiClient`
	Factory Method	`formatters.py`	`FormatterFactory`
	Abstract Factory	`environment_factory.py`	`EnvironmentFactory`
	Lazy Init	`socket_handler.py`	`get_transcriber()`
Estructural	Adapter	`audio_service.py`	`AudioProcessorPort`
	Facade	`socket_handler.py`	`register_socket_events()`
	Repository	`transcription_repo.py`	`TranscriptionRepository`
Comportamental	Strategy	`vad_service.py`	`VADStrategy`
	State	`vad_service.py`	`VADState`
	Observer	`socket_handler.py`	`@sio.event` handlers
	Dependency Injection	`transcriptions.py`	`Depends(get_service)`
	Template Method	`socket_handler.py`	`_process_pcm_chunk()`

Cuadro 2: Resumen de patrones de diseño implementados en SpeechNotes.

4.2. Adherencia a Principios SOLID

Principio	Evidencia
SRP	Cada archivo tiene una responsabilidad única: `audio_service.py` solo transforma audio; `vad_service.py` solo detecta voz; `transcription_repository.py` solo accede a datos.
OCP	Nuevos formatos de audio (Adapter), estrategias de VAD (Strategy) y formateadores (Factory) pueden añadirse sin modificar código existente.
LSP	`WebMAudioAdapter` y `PCMPassthroughAdapter` son sustituibles donde se espera `AudioProcessorPort`.
ISP	Las interfaces abstractas (`AudioProcessorPort`, `VADStrategy`, `OutputFormatter`) definen solo los métodos necesarios para sus consumidores.
DIP	Los módulos de alto nivel (`SocketHandler`, `TranscriptionService`) dependen de abstracciones, no de implementaciones concretas.

Cuadro 3: Adherencia a principios SOLID en el sistema SpeechNotes.

5. Discusión

5.1. Fortalezas Arquitectónicas

- La **separación por capas** (Presentation → Service → Repository) facilita el testing unitario y la mantenibilidad.
- El uso de **Abstract Factory** para entornos de transcripción permite cambiar de NVIDIA Riva a otro motor (e.g., Whisper local) sin modificar la lógica de negocio.
- El **Strategy Pattern** para VAD permite experimentar con diferentes algoritmos de detección de voz de forma transparente.
- La **Facade** de Socket.IO oculta toda la complejidad del pipeline de audio detrás de una interfaz limpia de eventos.
- La **Dependency Injection** vía FastAPI `Depends()` hace que los routers sean fácilmente testeables con mocks.

5.2. Áreas de Mejora

- **Caché distribuida**: La caché actual en `ApiClient` es en memoria (volátil). Considerar Redis o SWR para persistencia.
- **Implementar LocalBatchFactory**: Completar la fábrica para procesamiento por lotes con Whisper local.
- **Tests unitarios**: Aumentar la cobertura aprovechando la inyección de dependencias existente.
- **Logger centralizado**: Reemplazar `print()` por un sistema de logging estructurado (e.g., `loguru`).
- **Interfaz de VAD**: Considerar la adición de una estrategia basada en ML (e.g., Silero VAD) para mejorar la precisión.

6. Conclusiones

El sistema SpeechNotes